

PKMS Competitive Audit

Logseq OG · Logseq 2.0 DB · Tine · TyLog

Prepared 2026-08-20 · synthesis + 4 research legs · 12/12 adversarial spot-checks confirmed

The Logseq ecosystem split in 2026 into three diverging worlds, and none occupies TyLog's ground. **Logseq OG** is officially maintenance-only and observably idle. **Logseq 2.0 DB** is a real data-model upgrade shipped with beta regressions and a one-way, desktop-only lock-in answer. **Tine** is a two-month-old solo rewrite, fast and file-compatible but unproven. TyLog is the only one with typed, compilable plaintext as its storage layer, native typeset-PDF output, and release-grade Android sync safety — at the cost of no block references, no query language, and no plugin ecosystem.

SOLUTIONS COMPARED 4 OG, DB, Tine, TyLog	FEATURE ROWS SCORED 16 matrix, next page	SPOT-CHECKED CLAIMS 12 / 12 confirmed, 0 contradictions	RANKED TYLOG GAPS 7 see closing section
---	---	--	--

Verdict

TyLog does not compete with any of the three on their own ground — outliner depth, query power, plugin ecosystems. It competes on a different axis none of them hold: plaintext that is simultaneously structured data and a compilable typesetting program, paired with the field's most trustworthy mobile story. The strategic opening is real but narrow: OG's stranded users are an import target TyLog already partly serves, and the field's shared weak point — mobile reliability — is TyLog's strongest claim.

Feature matrix

Legend: ✓ full · 🟡 partial or scoped · ✗ absent · 🚫 deliberately out of scope. TyLog evidence = file paths in `docs/research-tylog-features.md`.

Row	Logseq OG (file)	Logseq 2.0 DB	Tine	TyLog
1 • Note model	✓ blocks + key:: text props, namespaces, aliases; regex-parsed	✓ "nodes"; typed first-class properties, classes w/ inheritance, closed values	✓ OG model reimplemented, file-compatible	✓ whole-note Typst files; typed-ish props, tags+synonyms, aliases
2 • Editor	✓ outliner, live-preview, KaTeX, md/org	✓ outliner; shadcn tables/views; org-mode dropped	✓ outliner parity + sheets/kanban, split panes	🚫 block-level edit, preview/source/split, magic actions; not an outliner
3 • Block refs / transclusion	✓ ((uuid)), {{embed}}	✓ unified [[]] refs + node embeds	✓ ((id)), {{embed}} parity	✗ note-level refs only, by design
4 • Tasks & scheduling	✓ markers, priorities, SCHEDULED/DEADLINE, org repeaters, LOGBOOK	✓ Task class, status history, repeater cookies	✓ OG parity + task carry-forward	✓ statuses/priorities/due+scheduled + RRULE recurrence, reminders, time tracking, deps
5 • Queries	✓ {{query}} DSL + raw Datalog	✓ visual query builder + rewritten advanced queries + Views	🚫 common-case DSL + visual builder; no raw Datalog	🚫 saved searches + report-block filters; no query language, by design
6 • Graph & backlinks	✓ graph view, linked+unlinked refs	✓ Graph View V2, rebuilt for performance	✗ no graph view yet; linked/unlinked refs ✓	✓ backlinks + 5 graph modes, community detection, LLM relink
7 • Search & indexing	🚫 Datascript rebuilt from files at startup; slow at scale	✓ fts5 trigram search-db, incremental	✓ Rust full-text, regex syntax, search-as-workspace	✓ worker-isolate gzipped index; roughly 17 min first index of 1,700 files on old Android hardware, then incremental
8 • Sync & collaboration	🚫 paid E2EE Sync (beta) or DIY file sync; single-writer; no RTC	🚫 RTC rebuild, E2EE default; invite-gated beta	🚫 no own backend; watcher + Syncthing-conflict merge UI	✓ Nextcloud WebDAV, ETag safety, manual conflict UI, background service; no E2EE/RTC
9 • Import / export	✓ EDN/JSON/OPML/Roam/HTML export; md corruption risk acknowledged	🚫 OG→DB migrator (lossy edges); EDN export; md export lossy by design	🚫 no import needed (same files); static HTML publish, per-page PDF	✓ Logseq+Obsidian import shipped, DB-EDN import planned; PDF export; no HTML

Row	Logseq OG (file)	Logseq 2.0 DB	Tine	TyLog
10 · Storage & data ownership	✓ plain md/org files are source of truth	× SQLite is source of truth; Markdown Mirror is one-way, desktop-only	✓ byte-compatible Logseq md files	✓ plaintext Typst files; disposable JSON indexes; every note compiles
11 · Plugins / extensibility	✓ JS plugin API + marketplace (desktop-only)	🕒 opt-in supportsDB (roughly 65 plugins); API bugs open; new CLI+MCP surface	🕒 own sandboxed WASM API; no Logseq-plugin compatibility	× none, by design; custom kinds/properties only; external article-pipeline as producer
12 · Whiteboards / SRS / PDF annotation	✓ / ✓ / ✓ — all native	× removed / 🕒 reimplemented, no data migration / 🕒 lossy import	🕒 external SVG round-trip / × / ✓ native, a differentiator	🕒 / 🕒 / ×
13 · Calendar / journals	✓ journals (docs are stubs)	✓ journal class, natural-language date input	✓ journals + agenda, calendar markers, templates	✓ month grid with markers, date refs, task dues
14 · Platforms & mobile	🕒 Electron + Android/iOS; officially tracked freezes, RAM evictions	🕒 desktop beta + web (OPFS); iOS invite-only, Android pre-alpha	🕒 Linux-first desktop; Android sideloaded APK; no iOS	✓ Android release-grade, real-vault tested; macOS release; iOS dev-only
15 · Publishing / typesetting	🕒 static SPA publish; no PDF	🕒 paid Sync-gated Publish at logseq.io	🕒 static HTML + per-page PDF	✓ native Typst→PDF: typeset notes, reproducible .typ + .pdf reports, bibliography/citations, math
16 · License / governance / momentum	AGPL; maintenance-only, idle roughly 3 months	AGPL; company-backed, nightly cadence, one tagged 2.0.x release	AGPL; solo maintainer, no external code PRs, near-daily releases, 2 months old	Proprietary/private; single developer, active

How each is implemented

Logseq OG

CLJS · Electron · mldoc · Datascript

The OCaml `mldoc` parser reads Markdown/Org into an in-memory Datascript database rebuilt from files at startup. Files remain the source of truth and the database is disposable. That buys ownership and git-ability, but costs startup/scale performance, leaves `key::` properties parse-fragile, and carries an explicit corruption warning from the maintainers.

Logseq 2.0 DB

CLJS · SQLite · fts5 · RTC side DB

Datascript now persists into SQLite: transit-encoded forked-Datascript index nodes in a `kvs` table, plus `fts5` search and `client-ops` RTC stores. Typed entity properties, classes, tasks, views, and bulk operations eliminate a class of text-parse bugs; the trade is an opaque, frequently changing on-disk format, lossy Markdown export, and an ecosystem reset.

Tine

Rust · Tauri · WebKitGTK · SolidJS

A from-scratch Rust core parses and indexes Logseq OG's exact files behind a fine-grained reactive frontend. Zero migration and byte-compatible files make it a reversible, fast bet. A solo, AI-assisted build with no external code-PR path also means plugin compatibility, graph view, SRS, and iOS are absent or explicitly out of scope.

TyLog

Flutter · Rust · Typst · WebDAV

Plaintext Typst notes are simultaneously readable text, structured `#tylog.note/task/ref-note` calls, and compilable programs. Worker-isolate indexing, ETag-safe Nextcloud sync, and competitor importers preserve OG-class ownership while adding compile-verifiable typed fields. It intentionally stops short of DB's entity graph and closed-value model.

Implementation consequence

The systems do not merely expose different features; their storage choices set different failure modes. OG and Tine optimize reversibility, DB optimizes entity semantics, and TyLog optimizes compile-checked documents plus dependable mobile file sync.

Pros & cons by solution

Logseq OG (file-based)

Dead end to invest in

PROS

- Maximal data ownership, plain files
- Mature full feature set: queries, whiteboards, SRS, PDF annotation, plugins
- Free DIY sync

CONS

- Frozen — security-only maintenance, observably idle roughly 3 months
- Acknowledged corruption risk; single-writer sync
- Poor large-graph performance; chronically rough mobile

Logseq 2.0 DB

Future, early adopters pay

PROS

- Best-in-field data model: typed properties, classes, closed values, views
- Performance at scale is a design goal
- Real RTC/E2EE sync substrate; company momentum (nightlies, CLI+MCP)

CONS

- Data lock-in is structural: one-way Mirror, lossy md export, EDN needs tooling
- Beta regressions: whiteboards gone, SRS not migrated, plugins reset to roughly 65
- Mobile still invite/pre-alpha; monthly schema churn under the hood

Tine

Promising, too young to depend on

PROS

- Fastest architecture in the field
- Zero-migration Logseq-file compatibility — a reversible bet
- Native PDF annotation + sheets/kanban that OG lacks; high shipping velocity

CONS

- Bus-factor-one, two months old, AI-assisted codebase, no external review path
- No plugin compatibility, graph view, SRS, or iOS
- No own sync backend

TyLog

Differentiated, real gaps

PROS

- Unique typeset-PDF axis: notes to publishable documents natively
- Strongest task engine: RRULE recurrence + reminders + time tracking
- Best mobile reliability posture: release-grade Android + sync-conflict safety
- Plaintext with verifiable structure — writers must compile; import from both Logseqs

CONS

- No block refs/transclusion or query language — a depth ceiling for outliner power users
- No extensibility story; no SRS, whiteboards, or PDF annotation
- Single-developer, closed source — same bus-factor critique as Tine applies
- No E2EE sync

TyLog gaps & opportunities, ranked

1. **Ship the Logseq DB importer.** Plan already written (`docs/superpowers/plans/2026-08-20-logseq-db-import.md`). OG users are stranded and DB users fear lock-in — TyLog can be the exit for both. Cheap: the plan exists.
2. **Position on “a typeset document, not a database.”** No competitor compiles a note to a publishable PDF natively. This is the only non-replicable moat in the matrix — lean on it in any public material.
3. **Query-lite, not a query language.** All three competitors have some query mechanism, but Tine proves a scoped DSL is enough for everyday use. TyLog's report blocks + saved searches cover most of it; the real gap is **inline** dynamic sections inside notes. Extend report blocks before ever considering a DSL.
4. **PDF annotation.** Tine's proven differentiator and a natural TyLog fit — the articles pipeline and reading mode already exist; annotations could be Typst data. The largest genuinely missing feature for reading-centric workflows.
5. **E2EE option for sync.** age is already in the backup design (`docs/age-encrypted-backup.md`); Logseq Sync uses age too. An age-encrypted WebDAV layer would close the one sync axis where Logseq DB leads.

6. **Block-level refs: keep saying no.** OG/Tine users expect them, but they are the single largest complexity driver in outliner codebases (UUID tracking, transclusion rendering, ref-integrity on edit). TyLog's note-level model and compile-checked writers are a deliberate simplicity moat — revisit only on real user demand.
7. **SRS/flashcards: defer.** Logseq DB broke its own users' SRS data in migration; Tine skipped it entirely. Demand signal is low.

Graphify relationships to the codebase

Path	What the graph establishes	Audit consequence
DB format → importer	Logseq DB-Version Graph Storage Format → Implementation Plan for Logseq DB EDN Importer → db_import → src/vault_import.rs (3 hops)	The highest-ranked gap already terminates at the shipped import boundary rather than requiring a parallel subsystem.
Matrix → importer	16-Row Feature Matrix → Logseq DB Feature Audit → this synthesis → DB-format research → importer plan → db_import → src/vault_import.rs (6 hops)	Competitive evidence, storage research, delivery plan, and implementation entry point form one traceable chain.
Opportunity links	Query-lite connects saved searches/full-text indexing; PDF annotation connects article ingestion and reading mode; the typeset moat connects the unique TyLog comparison axis.	Treat these as ranked product hypotheses, not extracted proof of implementation.

Graphify provenance: source-document edges are EXTRACTED; the links from opportunity nodes into code are explicitly INFERRED. The audit corpus added 105 nodes and 93 edges, including dedicated PKMS comparison communities.

Method note

Research ran as independent primary-source agents across five docs, each claim cited. A 12-claim adversarial spot-check (4 per doc) against cited URLs returned 12 of 12 confirmed, zero contradictions. TyLog rows come from code inspection, not marketing. Known thin spots inherited from the source research: DB sync pricing has no primary source, DB backlink-algorithm changes are undocumented, and OG journal mechanics are covered only by stub docs.

Research evidence appendix

Full source-faithful conversion of all four research legs

This appendix keeps the complete supporting research inside the same report rather than reducing it to a bibliography. All external links remain live. The first three legs carry primary-source citations; the TyLog leg is codebase-derived and cites repository paths.

Research leg	Evidence role	Preserved links
Logseq OG	Maintenance posture, feature baseline, file model, mobile and sync	44
Logseq 2.0 DB	Typed entity model, storage, migration, plugins, sync and beta state	110
Tine	Fork architecture, parity/divergence, governance, release velocity	13
TyLog	Code-derived current feature surface and implementation evidence	Repository paths

Verification gate

Twelve claims were adversarially spot-checked against their cited URLs — four per web-researched leg. Result: **12 confirmed, 0 contradictions**. The appendix preserves 167 live external evidence links in total; repeated citations remain repeated where the source uses them to support separate claims.

Logseq OG (file-based, pre-2.0) — Primary-Source Feature & Maintenance Audit

Scope. This document covers Logseq's original, file-based, markdown/org-mode product — retroactively named "**Logseq OG**" after Logseq split its codebase in 2026 — as distinct from the **Logseq 2.0** SQLite/DB-first rewrite. Research was conducted in **August 2026** against primary sources only: the `logseq/logseq` and `logseq/og` GitHub repositories (code, README, releases, issues), `docs.logseq.com` / the `logseq/docs` repository, `blog.logseq.com`, and official forum/announcement pages published by the Logseq team on `logseq.io` and `discuss.logseq.com`. Where no primary source could be found after a genuine search, this is stated explicitly rather than filled in from secondary sources.

Important context found during research: Logseq announced in 2026 that it is splitting into two products. The file-based, Markdown/Org-mode app is now maintained separately as "**Logseq OG**" at `github.com/logseq/og`, receiving security fixes and Electron/dependency upgrades but no new features, while the "Logseq" name and product roadmap now belong to the new local-SQLite, DB-first rewrite at `github.com/logseq/logseq`. Because `logseq/og` was forked from `logseq/logseq` at the point of the split, most of Logseq's historical documentation (in `logseq/docs`, and much of `docs.logseq.com`) still describes the shared, pre-split feature set and is treated here as describing Logseq OG except where a source explicitly marks a feature as DB-version-only (e.g. RTC sync, typed properties/classes, dashboard queries) — those are called out as **absent in OG**.

1. Note model (pages, blocks, properties, tags, namespaces, aliases)

Every page and every outline block can carry `key:: value` properties. Page properties live in the first block of a page (the "frontmatter" block); block properties live in any other block. Property names are case-insensitive, lower-cased, and `_` is auto-renamed to `-`. Property values can contain plain text, `[[page links]]`, and `#tags`; wrapping a value in quotes suppresses link parsing. Built-in properties like `tags` and `alias` also accept comma-separated lists (`tags:: motor, steering wheel`) which are auto-converted to page references. Namespaces are page titles containing `/` (e.g. `Logseq/Features`), stored on disk with an encoded filename (`Logseq___Features.md` in the modern `:triple-lowbar` format, or percent-encoded `Logseq%2FFeatures.md` in the legacy format). Aliases are declared with `alias:: PKM` in a page's first block, redirecting `[[PKM]]` references to that page.

- Properties: `logseq/docs:pages/Properties.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Properties.md>
- Built-in properties list: `logseq/docs:pages/Built-in Properties.md`, which also points to the canonical source `logseq/graph_parser/property.cljs` — https://github.com/logseq/logseq/blob/master/deps/graph-parser/src/logseq/graph_parser/property.cljs
- Filename encoding / namespaces: `logseq/docs:pages/Filename format.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Filename%20format.md>
- Aliases: `logseq/docs:pages/Aliases and external links.md`
- The dedicated `logseq/docs:pages/Namespace.md` and `logseq/docs:pages/Tags.md` pages are themselves unfinished TODO stubs in the official docs repo ("TODO Document this feature") — noted here as an honest primary-source gap, not filled in from secondary sources.

2. Editor (outliner, markdown/org-mode dual support, WYSIWYG-ish editing, math/katex, tables)

Logseq OG is a block-based outliner where every line is a block that can be collapsed, indented, and re-parented; the same graph can mix Markdown and Org-mode files, both parsed by Logseq's own parser library, `mldoc`. Editing is a live-preview ("WYSIWYG-ish") textarea that renders formatting (bold, links, embeds) around the cursor while a block is not focused. Inline and block math is supported via KaTeX syntax (`$...$ / $$...$$`), including chemistry via the `mhchem` package. Tables are a "versioned component": version 1 is the original plain markdown table; a version-2 beta adds per-table props (`logseq.table.hover`, `.compact`, `.stripes`, `.borders`, `.max-width`, `.color`) settable inline or via `config.edn`.

- Markdown syntax: `logseq/docs:pages/Markdown.md`
- Org-mode syntax and parser: `logseq/docs:pages/Org Mode.org` — both explicitly say "parsed by <https://github.com/logseq/mldoc>"
- Math/KaTeX: `logseq/docs:pages/Math Block.md`
- Tables: `logseq/docs:pages/Tables.md`

3. References (wikilinks, block refs, embeds, transclusion)

`[[page name]]` creates a page (wiki)link; `((uuid))` creates a block reference to a specific block by its UUID, created by typing `(`, using the `/Block` reference command, or copying a block-ref via a keyboard shortcut. `{{embed [[page name]]}}` transcludes an entire page's content into the current block; `{{embed ((block-uuid))}}` transcludes a single block and its children. Labeled variants exist: `[display text]([[page name]])` and `[display text]((block-uuid))`.

Referenced blocks show a reference counter that expands all linked references grouped by page.

- `logseq/docs/pages/Markdown.md` (syntax table for all reference types)
- `logseq/docs/pages/Block Reference.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Block%20Reference.md>

4. Tasks & scheduling

Tasks are just blocks with a workflow-marker keyword at the start. Two selectable marker flavors exist: `LATER` → `NOW` → `DONE` (default) and `TODO` → `DOING` → `DONE`, cycled with `Ctrl/Cmd+Enter` or typed directly (`/LATER`). Additional markers `CANCELED/CANCELLED`, `IN-PROGRESS`, and `WAIT/WAITING` can be typed manually. Three optional priorities, `[#A]/[#B]/[#C]`, attach via commands or by typing the bracket syntax after the marker. `SCHEDULED: <date>` and `DEADLINE: <date>` commands attach dates to any block (not just tasks), configurable to show upcoming items N days ahead via `:scheduled/future-days` in `config.edn`. Both support repeaters with three kinds: `.+` (repeats from last completion), `+` (keeps same weekday), `X` (repeats X units from original schedule) — e.g. `SCHEDULED: <2021-05-26 Wed 7:00 .+1d>`. A built-in time tracker / LOGBOOK-style clock feature exists and can be toggled off via a setting.

- `logseq/docs/pages/Tasks.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Tasks.md>

5. Queries (simple, advanced datalog, dynamic tables)

Two query tiers exist. **Simple queries** (`{{query ... }}`) use a small filter DSL with boolean operators `and/or/not` and filters including `between` (journal date ranges with symbols like `today`, `-7d`), `page`, `property`, `full-text query` (desktop-only), `task`, `priority`, `page-property`, `page-tags`, `all-page-tags`, and `sort-by`. **Advanced queries** are raw Datalog against the in-memory Datascript database, written as an EDN map (`:title`, `:query`, `:inputs`, `:view`, `:result-transform`, `:collapsed?`, `:group-by-page?`, `:rules`, etc.), with special query inputs like `:current-page`, `:today`, and relative-date tokens (`:-7d`, `:+1m`). Results can render as dynamic tables via `query-table:: true` plus query-properties, `query-sort-by`, `query-sort-desc` block properties.

- Simple queries: `logseq/docs/pages/Queries.md`
- Advanced queries: `logseq/docs/pages/Advanced Queries.md` — explicitly states "Advanced queries are written with Datalog and query the Datascript database"
- Query-table properties: `logseq/docs/pages/Built-in Properties.md`

6. Knowledge graph & backlinks

A whole-graph force-directed **Graph view** is reachable from the profile menu. **Linked references** appear at the bottom of a page for every block/page that references it, grouped by page. **Unlinked references** — occurrences of a page's name in text that were never turned into `[[links]]` — are shown in a separate toggleable section at the bottom of a page.

- Graph view: `logseq/docs/pages/Knowledge Graph.md`
- Unlinked references: `logseq/docs/pages/Unlinked References.md`
- Linked references: `logseq/docs/pages/Linked References.md` — this doc page is itself an unfinished TODO stub in the official docs repo; the behavior is documented instead via the Block Reference and Knowledge Graph pages above.

7. Search & indexing

On open, Logseq OG parses every Markdown/Org file in the graph folder into an in-memory **Datascript** database (an immutable Datalog store) that backs both the outliner UI and queries; this is the "rebuild from files" architecture. Search (`Cmd/Ctrl-K`) covers pages, blocks, files, and commands, with a `/` filter prefix; block search only finds blocks inside pages (not, e.g., the sidebar). Desktop additionally has full-text search across multiple blocks with out-of-order term matching. Search is backed by a separate, rebuildable search index (`Rebuild search index` command) distinct from the Datascript graph DB.

- Search: `logseq/docs/pages/Search.md`
- Datascript as the query engine/backing DB: `logseq/docs/pages/Advanced Queries.md` ("query the Datascript database"); Datascript is also listed as a core dependency in the `logseq/og` README credits section — <https://raw.githubusercontent.com/logseq/og/master/README.md>

- Frontend source directories consistent with this architecture (`/db`, `/persist_db`, `/search`, `/worker`) are visible in the repo tree: <https://github.com/logseq/og/tree/master/src/main/frontend>

8. Sync & collaboration

Logseq Sync is a paid, end-to-end-encrypted cloud sync add-on (in BETA at time of the cited doc), available to Open Collective backers contributing \$5–\$15/month, syncing up to 10 graphs across Desktop/Android/iOS. Each graph — including file names and paths — is encrypted client-side with a password using the `age` encryption tool; encrypted blobs are stored on AWS. Logseq Sync explicitly should not be combined with git or other third-party sync (iCloud/Syncthing/Dropbox) for the same graph. Separately, because graphs are just folders of plain-text files, any third-party file-sync tool (iCloud, Dropbox, Syncthing) works as a DIY sync mechanism, with the caveat that only one device should actively edit at a time to avoid conflicts. A built-in **Git Auto-Commit** feature can commit (but not push) local changes to a git repo at a configurable interval (1–600s) for version history/backup, independent of any sync mechanism. Real-time multi-user collaboration (RTC) is a DB-version-only feature and is **absent in OG** — the OG-era docs explicitly note "This isn't supported yet" for using Sync with other users.

- Logseq Sync: `logseq/docs:pages/Logseq Sync.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Logseq%20Sync.md>
- File-sync-as-DIY-sync and single-writer caveat: `logseq/docs:pages/How to sync your Logseq graph across devices.md`
- Git Auto-Commit: `logseq/docs:pages/Git Auto-Commit.md`
- RTC as DB-version-only: `logseq/og:README.md` ("The DB version also has a new sync approach, RTC...") — <https://raw.githubusercontent.com/logseq/og/master/README.md>

9. Import/export

Export: supports whole-graph export (EDN, JSON, Standard Markdown, OPML, **Roam JSON**, HTML) and single-page/block/selection export (Text, OPML, HTML, PNG), each with format-specific options (strip brackets/emphasis/tags, block-depth limit, indentation style, include/exclude properties). Note block properties are explicitly dropped from "Standard Markdown" graph exports.

- `logseq/docs:pages/Export.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Export.md>

Import: the official `logseq/docs:pages/Import.md` page is an unfinished stub ("TODO Document feature"), so Logseq's own docs do not spell out the Roam Research import flow in detail. Community/secondary sources (nesslabs.com, hub.logseq.com) describe a Roam JSON-export importer, but per this audit's sourcing rules that is **unverified against a primary source** beyond the fact that "Roam JSON" is a listed graph-export *output* format (see `Export.md` above), which implies round-trip compatibility exists at the format level. The presence of a Roam-format export option is primary-sourced; a dedicated Roam *importer* UI is referenced only secondarily and is flagged here as **unverified — no primary source found** for its exact mechanics.

logseq-publish static export: see row 15.

10. Storage format & data ownership

Every page is one plain-text file (`.md` by default, `.org` if configured) inside a `pages/` (and `journals/`) folder on local disk; namespace pages are encoded into filenames (row 1). No proprietary database sits between the user and their notes — the Datascript DB (row 7) is a disposable in-memory index rebuilt from the files, not the source of truth. The OG-era README is explicit that this brings a tradeoff: "When using file graphs, **data corruption is possible** as some file content can be duplicated. We only recommend using it with file graphs if you make regular backups with git."

- `logseq/og:README.md` — <https://raw.githubusercontent.com/logseq/og/master/README.md>
- Filename encoding mechanics: `logseq/docs:pages/Filename format.md`

11. Plugins/extensibility

A desktop-only plugin API lets third-party JS/TS plugins extend the app; plugins (and themes, which are distributed as a plugin type) are browsed and installed from an in-app **Marketplace** dashboard (`tp` shortcut) backed by github.com/logseq/marketplace, or side-loaded via "Load unpacked plugin." Plugin updates are checked every 12 hours. Plugin API docs live at `plugins-doc.logseq.com`. Theming can also be done without plugins via `logseq/custom.css` (per-graph, loaded at startup — the docs explicitly warn about self-maintaining any custom CSS for stability/security) and `logseq/custom.js`. Graph-level behavior is configured via `logseq/config.edn`, documented informally via a template file in the source tree rather than a full reference page. Plugins are explicitly **desktop-only** — "not available for mobile or the browser."

- `logseq/docs/pages/Plugins.md`, `logseq/docs/pages/Marketplace.md`, `logseq/docs/pages/custom.css.md`, `logseq/docs/pages/config.edn file.md`
- `config.edn` reference source: `logseq/logseq:deps/common/resources/templates/config.edn` — <https://github.com/logseq/logseq/blob/master/deps/common/resources/templates/config.edn>
- Marketplace repo: <https://github.com/logseq/marketplace>

12. Whiteboards, flashcards/SRS, PDF annotation

Whiteboards: a toggleable spatial-canvas feature ("free for everyone") built on "a fork of tldraw," reachable from a dedicated left-sidebar section. Each whiteboard is stored as its own `.edn` file in a `whiteboards/` folder inside the graph — deletable directly from the filesystem. The canvas supports drag-and-drop page/block embeds ("portals"), images/PDFs/videos, YouTube/Tweet-aware object embeds, freeform drawing/shapes, and a toolbar (Select, Move, Portal, Pencil, Highlight, Eraser, Connector, Text, Shapes).

Flashcards/SRS: any block tagged `#card` or `[[card]]` becomes a flashcard (can include Clozes). Cards are reviewed via a dedicated "Flashcards" sidebar tab (t c shortcut) or scoped with `{{cards [[Page]]}}` / `{{cards (not [[Page]]}}` queries. Scheduling state is stored as ordinary block properties on the card block itself: `card-last-interval`, `card-repeats`, `card-ease-factor`, `card-next-schedule`, `card-last-reviewed`, `card-last-score` — an SM-2-family (SuperMemo-derived) spaced-repetition scheduler, evidenced by the ease-factor/interval property shape and an explicit doc link to SuperMemo's SM5 algorithm page; the docs do not name the exact algorithm variant in prose, so "SM-2-family" is an inference from the property names and cited link, not a verbatim primary-source label.

PDF annotation: desktop-only. PDFs are dragged into a block or uploaded via `/upload` an asset, stored inside the graph's `assets/` folder (with no automated cleanup if unlinked). Users can highlight selected text or drag-select an area, each with a chosen color; highlights can be pasted as a reference into any block. Three PDF-viewer dark-mode themes and an outline/TOC view are supported.

- Whiteboards: `logseq/docs/pages/Whiteboard.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Whiteboard.md>
- Flashcards: `logseq/docs/pages/Flashcards.md` — <https://raw.githubusercontent.com/logseq/docs/master/pages/Flashcards.md> (links to SuperMemo's SM5 page and "Augmenting Long-term Memory")
- `logseq/docs/pages/Spaced Repetition.md` is a stub that only quotes Wikipedia's definition of spaced repetition — the algorithm itself is not documented in prose by Logseq; flagged as a primary-source gap.
- PDF highlights: `logseq/docs/pages/PDF highlights.md`

13. Calendar/journals

The dedicated `logseq/docs/pages/Journals page.md` doc is an unfinished TODO stub in the official docs repo, so Logseq's own documentation does not spell out journal-page mechanics in prose beyond referencing an external "Getting started with the Journals page" tutorial page (also present in the docs repo but not fetched in full here). What is directly confirmed from primary sources: journal pages are a first-class page type distinguishable from regular pages in Datalog queries (the `between` simple-query filter explicitly operates "only on blocks on the journal pages," and advanced queries have dedicated relative-date inputs like `:today`, `:-7d`, `:+1m` "useful for querying journal pages"), and a `setting/enable journals` toggle exists to turn the feature off entirely. Per-day settings such as journal page title/file format live in `config.edn` (not itself examined page-by-page here).

- `logseq/docs/pages/Queries.md` (between filter description) and `logseq/docs/pages/Advanced Queries.md` (relative-date inputs)
- `logseq/docs/pages/setting___enable journals.md` (existence of the toggle, filename indicates a settings-reference stub page)
- `logseq/docs/pages/Journals page.md` — TODO stub, flagged as a primary-source documentation gap.

14. Platforms & mobile quality

Logseq OG ships as an Electron desktop app (Windows/macOS/Linux, with a Linux install script) and mobile apps for Android and iOS, all built from the same file-based codebase/graph format, per the `logseq/og` README and its release list (e.g. "Desktop/Android APP 1.0.0"). Mobile quality has been a persistent, officially tracked pain point: the `logseq/logseq` GitHub issue tracker (the pre-split, shared-codebase repo, which mobile issues were filed against before the 2026 split) contains numerous maintainer-triaged reports, including app-restart/RAM pressure on Android (issue #5638), long/repeated loading screens on iOS (#10028) and Android (#10119), the iOS app freezing (#9615), the Android app freezing when opening search (#10771), and slow syncing causing disruption (#10999). These are cited as evidence of documented, officially-tracked mobile pain points rather than as maintainer statements acknowledging the problems in prose.

- Releases (desktop + Android under one artifact): <https://github.com/logseq/og/releases>

- Mobile issues (official tracker, `logseq/logseq`):
 - <https://github.com/logseq/logseq/issues/5638> ("App restarting regularly. Not enough RAM to keep it in background")
 - <https://github.com/logseq/logseq/issues/10028> ("iOS app 'loading...' screen appears 6-8 times before I can use the app")
 - <https://github.com/logseq/logseq/issues/10119> ("Opening the Android app (almost) always takes a long time")
 - <https://github.com/logseq/logseq/issues/9615> ("iOS app keeps freezing")
 - <https://github.com/logseq/logseq/issues/10771> ("opening search on Android client freezes app")
 - <https://github.com/logseq/logseq/issues/10999> ("Slow syncing leading to disruption")

15. Publishing/typesetting

logseq-publish: a graph can be exported as a static, read-only single-page app via "Export public graph pages as html" (desktop-only). Publishing is controlled per-page via a `public` property (or a graph-wide "public by default" setting with per-page opt-out), and the exported bundle can read the graph's `config.edn`, `custom.css`, `custom.js`, and `export.css`. Most read-only features (search, page/block links) work in a published app; anything requiring editing, or desktop-only features like plugins/themes-as-plugins, does not — the docs give a `custom.css` `@import` workaround for applying a theme to a published site. Publishing/export tooling itself is decoupled into a companion GitHub Action/CLI, `logseq/publish-spa`. No PDF-export quality claims are made in the official docs beyond PNG image export of a page/selection/whiteboard; a dedicated "export whole graph to PDF" feature is **absent** from the primary sources found — publishing targets HTML/SPA output, not PDF.

- `logseq/docs:pages/Publishing.md`, `logseq/docs:pages/Publish Web.md`, `logseq/docs:pages/Publishing (Desktop App Only).md`
- Publish action/CLI: <https://github.com/logseq/publish-spa>
- Export formats (no PDF listed): `logseq/docs:pages/Export.md`

16. Licensing, governance, maintenance

Both `logseq/logseq` (current DB-version repo) and `logseq/og` (file-version repo) ship a `LICENSE.md` containing the **GNU Affero General Public License v3.0 (AGPL-3.0)** verbatim.

Logseq the company was founded in 2020 by Tienso Qin (CEO), ZhiYuan Chen, and Huang Peng, and raised a \$4.1M seed round in May 2022 led by Patrick Collison (Stripe), Nat Friedman (ex-GitHub), and Tobias Lütke (Shopify), with participation from Sriram Krishnan (a16z), Craft Ventures, Matrix Partners China, Day One Ventures, Charlie Cheever, and Dave Winer; the team was nine people at the time of that post. The project also runs an Open Collective for community sponsorship/backing, separate from the equity raise, which directly funds the Logseq Sync beta (row 8).

The 2026 split, in the maintainers' own words (official announcement page, published on Logseq's own site at `logseq.io`): Logseq is dividing into two products — **Logseq OG**, the Markdown/file-based graph app moved to its own repo at `github.com/logseq/og`, and **Logseq**, the new local-SQLite database app which keeps the name and the forward roadmap. The stated reason: maintaining two distinct architectures inside one app meant "every feature, bug fix, and UX improvement must be built twice," slowing development. For Logseq OG specifically, the announcement commits to **"security fixes and patches"** and **"Electron and dependency upgrades"** — i.e. stability/reliability maintenance, explicitly **not** new feature development — with no forced migration: existing Markdown users can keep using OG indefinitely, and both apps can be run side by side.

What that maintenance has looked like in practice, checked directly against the repo (as of this research, August 2026): `logseq/og` was created 2025-12-25 as the fork point; its only tagged release is 1.0.0 on 2026-04-15; the most recent commits are dated **2026-05-28**, and are exactly the kind of maintenance promised — "Upgrade Electron to 41.7.1" and "fix: upgrade dugite 2.7.1 → 3.2.2 to resolve CVE-2023-5678" — alongside a batch of rebranding commits (custom URL scheme/deeplink changed to `logseq-og`, app-menu title changed to "Logseq OG", separate mobile app identifiers). The repo has had **no commits and no new release for roughly three months** at the time of this research (last push 2026-05-28), while `logseq/logseq` (the DB version) has nightly builds as recently as 2026-08-19 and shipped its 2.0.1 beta on 2026-07-13. This is a directly observable, primary-source data point for how "maintenance-only" is playing out in practice, worth flagging for the competitive audit: the promise (security/Electron patches, no new features) has been kept in kind but the observed cadence has gone quiet for months, not because of new source we could find describing a wind-down, but because there's simply no recent activity to show — noted here as an observation from repo data rather than an official statement of reduced commitment.

- License: <https://raw.githubusercontent.com/logseq/og/master/LICENSE.md> (and equivalently `logseq/logseq:LICENSE.md`)

- Company/funding: `blog.logseq.com` — <https://blog.logseq.com/logseq-raises-4-1m-to-accelerate-growth-of-the-new-world-knowledge-graph/>
- Split announcement (official, published on `logseq.io`): <https://logseq.io/page/b2ad9ce1-9cb7-4436-8083-54cb4516d324/df4dc09d-0a12-4c87-904e-22a9bf4c350a> — "Big update: Logseq is splitting into two versions"
- `logseq/og` repo metadata (created/pushed dates, release list, commit log) via the GitHub API: <https://github.com/logseq/og>, <https://github.com/logseq/og/releases>, <https://github.com/logseq/og/commits>
- `logseq/logseq` nightly/beta release cadence for comparison: <https://github.com/logseq/logseq/releases>
- Official "why the database version" statement, including the (pre-split) commitment to keep supporting file-based graphs: <https://discuss.logseq.com/t/why-the-database-version-and-how-its-going-to-26744>

Implementation stack summary

- **Frontend:** ClojureScript. The `logseq/og` README credits "Clojure & ClojureScript — A dynamic, functional, general-purpose programming language" as a foundational dependency, and the repo's `src/main/frontend/` tree (handler, components, format, worker, etc.) is ClojureScript source. — <https://raw.githubusercontent.com/logseq/og/master/README.md>, <https://github.com/logseq/og/tree/master/src/main/frontend>
- **In-memory DB rebuilt from files: DataScript** — "An immutable database and Datalog query-engine for Clojure, ClojureScript and JS" — is listed as a core dependency, and the Advanced Queries docs confirm queries run directly against "the Datascript database." The repo has `/db`, `/persist_db`, and `/worker` directories consistent with parsing files into this DB (worker isolate) rather than a persistent server-side store. — same README; `logseq/docs:pages/Advanced Queries.md`
- **Document parsing:** a separate OCaml/Angstrom-based parser library, `mldoc`, parses both Markdown and Org-mode into Logseq's internal AST — explicitly credited in the README and cited directly in the `Markdown.md` and `Org Mode.org` doc pages. — <https://github.com/logseq/mldoc>, `logseq/docs:pages/Markdown.md`, `logseq/docs:pages/Org Mode.org`
- **Git support: isomorphic-git**, "A pure JavaScript implementation of Git for NodeJS and web browsers," is credited as powering the Git Auto-Commit feature. — `logseq/og:README.md`
- **Desktop shell: Electron** — confirmed directly by the maintenance commit "Upgrade Electron to 41.7.1" in the `logseq/og` commit log, and by the split announcement's promise of "Electron and dependency upgrades" for OG. — <https://github.com/logseq/og/commits>, split announcement (row 16)
- **Mobile shell:** primary sources found do **not** name a specific mobile shell technology (e.g. Capacitor) for the file-based OG mobile apps — the `logseq/og` repo does contain a `/mobile` directory and mobile-specific commits (separate app identifiers, deeplink scheme), but no doc or README text was found stating the mobile wrapper framework by name. Flagged as **unverified — no primary source found** rather than assumed to be Capacitor.
- **Plugin runtime:** a JS/TS plugin API (desktop-only), documented separately at `plugins-doc.logseq.com` and distributed via the `logseq/marketplace` repo. — `logseq/docs:pages/Plugins.md`

Honest pros/cons of the file-based architecture

Strengths, per primary sources:

- **Data ownership / no lock-in:** every page is a plain `.md/.org` file on local disk; the Datascript DB is explicitly a rebuildable index, not the source of truth. The `logseq/og` README states the goal of user control directly, linking to the FSF's definition of free software. — <https://raw.githubusercontent.com/logseq/og/master/README.md>
- **Interoperability:** graphs can be created from pre-existing Markdown files, and export supports OPML, HTML, Roam JSON, and "Standard Markdown" for use in other tools. — `logseq/docs:pages/Export.md`
- **Sync-tool agnosticism / git-friendliness:** any file-sync tool works because the data is just files, and a built-in Git Auto-Commit feature turns any graph into a versioned git repo on a timer. — `logseq/docs:pages/Git Auto-Commit.md`, `logseq/docs:pages/How to sync your Logseq graph across devices.md`

Weaknesses, per primary/maintainer sources:

- **File corruption risk on concurrent/multi-device use:** the OG README itself warns "data corruption is possible as some file content can be duplicated," recommending regular git backups for file graphs — a maintainer-acknowledged weakness, not a third-party complaint. — <https://raw.githubusercontent.com/logseq/og/master/README.md>
- **Single-writer constraint:** official docs state only one device should actively edit a graph at a time when relying on generic file sync, requiring manual re-indexing after switching devices. — `logseq/docs:pages/How to sync your Logseq graph across devices.md`

- **Performance degradation on large graphs**, since the whole graph is parsed into an in-memory Datascript DB at startup: officially tracked GitHub issues document freezes/lockups on large graphs and pages — e.g. a graph of 18,500 interlinked pages freezing the graph view (#8398), an import of ~4,000 org journals with tens of thousands of blocks locking up indefinitely (#8544), 10–20 second UI freezes editing large files (#6002), and app launch failures once a graph exceeds ~300MB / 30,000 files (#11236).
 - <https://github.com/logseq/logseq/issues/8398>, <https://github.com/logseq/logseq/issues/8544>, <https://github.com/logseq/logseq/issues/6002>, <https://github.com/logseq/logseq/issues/11236>, <https://github.com/logseq/logseq/issues/8137>
- **key:: value properties live inside the text body**, not as structured metadata in a separate layer — this is the row-1/row-10 architecture itself (properties are literally text lines inside a block, delimited by newlines, per `logseq/docs:pages/Properties.md`: "a property value can't have newlines" is called out as a direct consequence of this embedding). The maintainers' own rationale for building the DB version cites this class of limitation in general terms — poor performance with large graphs, unreliable undo, lack of real structure for collaboration — as motivation for moving away from the file/property-in-text model. — `logseq/docs:pages/Properties.md`; <https://discuss.logseq.com/t/why-the-database-version-and-how-its-going/26744>
- **Markdown/Org parse fragility surfaces at the filename layer too**: Logseq had to ship a breaking, opt-in filename-format migration (`:legacy` → `:triple-lowbar`) specifically because encoding page titles (namespaces, special characters) into cross-platform-safe filenames was ambiguous and error-prone under the original scheme — an architecture-level cost of the one-file-per-page model that a DB does not have. — `logseq/docs:pages/Filename format.md`
- **Mobile quality**: see row 14 — multiple officially tracked, unresolved issues around freezing, slow loading, and background-memory eviction on both Android and iOS predate and are independent of the 2.0 split.
- **Maintenance-only status going forward**: by the maintainers' own commitment (row 16), OG will not receive new features, only security/Electron patches — a structural, permanent ceiling on the file-based product's capability relative to the DB version, confirmed by the observed ~3-month commit gap since the last Electron/CVE patch as of this research.

Rows/claims marked "unverified — no primary source found" or "absent" in this document: row 9 (Roam import mechanics beyond the Roam-JSON export format), row 13 (journal-page mechanics beyond the query-input evidence — the official doc page is an unfinished stub), row 14/Implementation-stack (mobile shell framework name), row 5/13 boundary (RTC collaboration is confirmed absent in OG, being DB-version-only).

Logseq 2.0 "DB version" — feature-surface competitive audit (as of Aug 2026)

Scope: what changed in the **feature surface** of Logseq's new database-backed product vs. the classic file-based product ("Logseq OG"), for a competitive audit against TyLog. Desktop beta **2.0.1** shipped 2026-07-13; this document reflects the state of `logseq/logseq` master and the `2.0.1` tag as researched 2026-08-20.

Method: primary sources only — the `logseq/logseq` GitHub repo (master + `2.0.1` tag), its `docs/` tree and ADRs, the sibling `logseq/docs` repo (`db-version.md`, `db-version-changes.md`, `og_import_graph_cases.md`), the `logseq/db-test` and `logseq/marketplace` repos, GitHub Releases, and official posts on `logseq.io` and `discuss.logseq.com`. Community forum/Reddit claims are explicitly labeled **COMMUNITY SENTIMENT** with an attributed source; nothing here is invented or inferred without a cited basis.

Storage internals (SQLite `kvs` schema, transit encoding, EDN export, schema version 65.33, `datascript` fork) are **not** re-derived here — see the sibling doc: [docs/research-logseq-db-format.md](#). Facts from that doc are linked, not re-researched.

TL;DR

- Logseq forked itself in mid-2026: the markdown/file product is now **Logseq OG** (its own repo, `github.com/logseq/og`, security-fixes-only going forward), and "Logseq" now means the SQLite/`datascript` **DB version**, which is where all new feature work happens ([split announcement](#)).
- The DB version unifies pages and blocks into "nodes," makes properties and tags first-class typed entities, and replaces TODO/DOING text markers and `{{query}}` macros with a Task class and a visual query/view builder — a genuine data-model upgrade, not just a storage swap ([docs/db-version.md](#)).
- Two headline features were **cut** at 2.0.1: whiteboards (removed outright, "hopefully" a future plugin) and, in effect, native canvas/Excalidraw ([db-version-changes.md](#)). Flashcards were reimplemented (old SRS data not imported) and PDF annotation changed shape; both still have open bugs as of Aug 2026 ([db-test issues](#)).
- Sync is being rebuilt as real-time collaboration (RTC) with a dedicated `client-ops-db.sqlite` and optional-but-default E2EE ([ADR 0003](#)), and is still beta/invite-gated for RTC as of Aug 2026 (community reports).
- The team's direct answer to the plaintext-lock-in complaint, shipped 2026-05/06, is the **Markdown Mirror**: a one-way, Electron-only, debounced markdown projection of the DB graph onto disk, explicitly *not* a full export and *not* editable externally ([ADR 0016](#)).
- Plugin API and marketplace compatibility for DB graphs are real but immature: a `supportsDB/supportsDBOnly` manifest flag exists in the marketplace repo, but multiple plugin-API calls have open bugs in `db-test` as of Aug 2026.
- Only **one** 2.0.x release exists on GitHub Releases as of 2026-08-20 — `2.0.1` (2026-07-13) — with a `nightly` unstable channel (latest `20260819`) carrying ongoing DB-version work; there is no 2.0.2 yet.

1. Note model (nodes, typed properties, classes/tags, closed values)

The storage-level model (everything is a `:block/uuid "node"`; properties as first-class `:logseq.class/Property` entities; classes via `:logseq.class/Tag + :logseq.property.class/extends`) is covered in the sibling doc's [§5](#) — reference that for the `datascript`-level facts.

User-facing additions not in the storage doc:

- "A node is a new term for a page or block because the two now behave similarly" — blocks can become pages by adding the `#Page` tag, and pages are disambiguated by tag rather than forced-unique names (e.g. "Apple #Company" and "Apple #Fruit" can coexist) ([docs/db-version.md](#)).
- Property schema editing is in-app: six user-facing property types (Text, Number, Date, DateTime, Checkbox, URL — plus Node/ref and Asset per the storage-doc's `:logseq.property/type` enum), each configurable with a default value and a **choices** list — this is the closed-values UX: a property can be restricted to a picker of pre-defined values, surfaced as chips/dropdown rather than free text ([docs/db-version.md](#)).
- Tags carry **Tag Properties** that every tagged node inherits ("Tags can have Tag Properties which are properties that all nodes inherit from a tag"), i.e. class-level default schema; tags support multiple parents via an "Extends" property and bidirectional properties ([docs/db-version.md](#)).
- Page-level properties are now set by editing the page **title** block rather than the first block of the page; block properties are edited inline in block content ([db-version-changes.md](#)).

2. Editor (property UI, tag-as-class UX, tables/views)

- Inline tagging changed interaction: typing # and pressing Enter now opens "a powerful tags feature" (tag creation/search modal) instead of instantly inserting a tag; inline tag *entry* now requires `Cmd-Enter`, and tags render to the block's right rather than inline ([db-version-changes.md](#)).
- Tables were rebuilt on `shadcn` (a React/Radix component set) with inline spreadsheet-style cell editing, replacing the old markdown-table rendering ([db-version-changes.md](#)).
- / slash commands were consolidated — "Advanced Commands" merged into the standard command list — and markdown syntax is no longer shown/editable as raw text in the block (e.g. heading level is set via a right-click menu, not by typing #s) ([db-version-changes.md](#)).
- New **Views** mechanism sits over any tag/class collection with three layouts — Table, List, Gallery (community FAQ additionally lists Kanban and Calendar view types; only Table/List/Gallery are documented in the primary `db-version.md`) — each with filter/sort/group and bulk multi-select actions (retag, bulk property edit, bulk delete) ([docs/db-version.md](#)).
- Templates are now created via a `#Template` tag rather than a property-based marker ([db-version-changes.md](#)).
- Org-mode file format support was dropped outright ("Org mode files are no longer supported") and the built-in Zotero integration was removed ([db-version-changes.md](#)).

3. References (block refs, embeds)

- References are unified: both page and block references use `[[ ]]` syntax now; the old block-embed parenthetical syntax `(( ))` is gone, and node embedding uses the same `/Node embed` command for both blocks and pages ([db-version-changes.md](#)).
- Namespace references (`[[foo/bar/baz]]`) still work for hierarchical page creation, but the created page's **stored name no longer embeds the namespace path** — this is presented as a fix (renaming a parent no longer cascades and breaks children) but is a real behavioral change from OG, where the full path was literally the page name. Namespace structure is now edited explicitly via a "Library" page, and the old `{{namespace}}` query macro is deprecated in favor of it ([db-version-changes.md](#)).
- At the storage layer, refs resolve through `:block/refs` / entity title per [the sibling doc](#), not raw text pattern matching — this is why old regex-based ref parsing from OG can't carry over.

4. Tasks & scheduling

Confirmed at the storage layer (Task class, status/priority/scheduled/ deadline as properties, no TODO/DOING text markers) in [the sibling doc](#). User-facing:

- Tasks are created via commands (`/todo`, etc.) rather than typing marker text; "Logbook timestamps have been replaced with Status change history" — i.e. task-state transitions are now tracked as structured property history rather than a `:LOGBOOK:` text block ([db-version-changes.md](#)).
- **Repeaters exist and are documented in detail**, adapted from org-mode's three repeater-cookie semantics, keyed on a new `:logseq.property.repeat/repeat-type` property:
 - `.+` — next occurrence = completion date + one interval (habit-style, resets from when you actually finished it).
 - `++` — next occurrence = original scheduled date + intervals, advanced until it's in the future (keeps a fixed weekday/anchor; this is the **default** for tasks without an explicit cookie).
 - `+` — next occurrence = original date + exactly one interval, can stack into overdue (fixed-date obligations like rent). Implementation lives in `src/main/frontend/worker/commands.cljs`. ([docs/recurring-tasks.md](#)).
- The 2026-05-16 team update reports "Task repeater cookies (`.+`, `++`, `+`) now behave according to documentation specifications," implying repeaters were buggy pre-May-2026 and were stabilized before the 2.0.1 beta cut ([discuss.logseq.com](#), "What's New with Logseq DB — May 16th 2026").

5. Queries

- The `{{query}}` inline macro and old advanced-query block syntax are gone as the primary UX. **Simple queries** are now created via a `/Query` slash command that opens a visual query builder; the builder writes text using internal entity IDs, supports a custom title, and — notably — "runs against all nodes instead of forcing the user to choose between blocks or pages" (a real semantic upgrade over OG's block-vs-page split) ([db-version-changes.md](#)).
- Filter renames reflect the property model: `(page-tags) → (tags)`, `(page-property) → (property)`, `(priority A) → (priority high)`; `all-page-tags` and `sort-by` filters are removed (sorting now happens via the Table view instead of a query filter) ([db-version-changes.md](#)).

- **Advanced queries** still exist as a feature but are edited as syntax-highlighted code blocks and require rewriting against the new schema: `:block/marker→:logseq.property/status`, `:block/priority→:logseq.property/priority`, `:block/deadline/:block/scheduled→ property equivalents`, `:block/content/:block/original-name→:block/title`; `:block/journal?`, `:block/left`, `:block/path-refs` are removed attributes; `:title`, `:group-by-page?`, `:collapsed?` options are deprecated ([db-version-changes.md](#)).
- The new **Views** mechanism (§2) is effectively "queries as a first-class UI object over a tag/class," layered on top of query results with Table/ List/Gallery rendering and bulk actions ([docs/db-version.md](#)).

6. Knowledge graph & backlinks

- Backlink/linked-reference mechanics inherit directly from the unified `[[ ]]`-reference model (§3) — no separate primary-source doc describes a behavioral change to the backlinks panel itself beyond that unification.
- The graph *visualization* was rebuilt: "Graph View V2 — A complete rebuild addressing performance issues... renders faster, scales better, and is easier to move around in," plus the ability to zoom into a task node from the graph ([discuss.logseq.com](#), "What's New with Logseq DB — May 16th 2026").
- The public roadmap separately lists "new graph visualization with improved performance" as an ongoing roadmap item, consistent with Graph View V2 being iterative rather than finished ([logseq.io public roadmap](#)).
- No primary source found describing algorithmic changes to backlink discovery itself (e.g. ranking, unlinked-reference detection) beyond the reference-model unification — absent as of Aug 2026.

7. Search & indexing

Per the sibling doc: a per-graph `search-db.sqlite` with an `fts5` trigram index over `blocks(id, title, page)`, kept live via `AFTER INSERT/UPDATE/ DELETE` triggers, derived/rebuildable data excluded from backups — see §1 "Side tables / side files" of the sibling doc. Not re-derived here.

User-facing addition: the search modal now shows recently-updated pages by default when opened with no query, and supports creating a new tag directly from the search modal ([db-version-changes.md](#)). The "All Pages" screen was renamed "Pages" and gained a table/list view toggle ([db-version-changes.md](#)).

8. Sync & collaboration

- The DB version's sync is being rebuilt as **RTC (Real-Time Collaboration)**, not the old file-diff sync. Client-side sync state is tracked in a dedicated `client-ops-db.sqlite` per graph (see the sibling doc's §1), backed by ADRs describing a Node.js sync-server adapter ([ADR 0001](#)), a `sync_meta/client_ops` split with `(created_at, id)`-ordered pending uploads ([ADR 0015](#)), entity-checksum reconciliation, and op-driven client rebase for conflicts ([ADR 0006](#), [ADR 0010](#) — titles only, not deep-read here: [docs/adr/](#)).
- **Encryption**: E2EE is the *default*, but is explicitly made **optional at graph-creation time**, and that choice is then immutable for the graph's lifetime — "The selected mode is stored as graph metadata and treated as immutable for that graph after creation." The stated rationale for allowing plaintext sync is self-hosted/trusted-infrastructure use and third-party tool integration that needs direct storage access ([ADR 0003](#)). A `:logseq.kv/graph-rtc-e2ee?` graph-level flag records the mode.
- **Self-hosting**: the ADRs and repo confirm a self-hostable sync-server adapter exists ([ADR 0001](#)); the official public roadmap separately lists "self-hosted sync" as a roadmap line item, and a forum thread specifically about self-hosting sync exists ([discuss.logseq.com](#), "Logseq Sync Self-Hosted possibility") — the team-vs-community split of that thread was not verified line-by-line here, so treat self-hosting maturity claims from it as **COMMUNITY SENTIMENT** unless corroborated by the ADR.
- **Status as of Aug 2026**: the official roadmap and the split announcement both describe DB-graph sync as beta/rolling-out — the split post says "Logseq Sync" for DB graphs is invite-based and encrypts "locally on your device" before upload, each device keeping "a local copy of your graph" (local-first posture retained) ([logseq.io](#), "Big update: Logseq is splitting into two versions"). The May 2026 team update lists concrete sync reliability fixes shipped — "RSA key caching, websocket recovery, and better handling of encrypted graphs and multi-device conflicts" ([discuss.logseq.com](#), May 16 2026 update) — consistent with sync still being actively stabilized post-2.0.1.
- **Pricing**: no primary-source pricing figure for DB-graph Sync/RTC or the mentioned "Logseq Pro" was found on [logseq.io](#) or in the repo as of Aug 2026 — the only concrete numbers found (Backer/Sponsor Open Collective tiers) come from third-party pricing-aggregator blogs, not Logseq itself; **no primary source found** for current DB-version sync pricing.
- **Multi-user/RTC status**: "Real Time Collaboration (RTC)" is named explicitly as a roadmap/beta item alongside the DB version and a new mobile app; the primary roadmap doc lists "conflict resolution when multiple clients

edit same content" and "recycle to restore deleted pages" as in-progress, not shipped-and-final, items ([logseq.io public roadmap](#)).

9. Import/export

Export mechanics (SQLite copy, zip, EDN :graph/:graph-human, lossy markdown export, debug transit, no JSON graph export) are covered in the sibling doc's §2 — reference that; this section adds fidelity/known-issue detail.

- **File-graph → DB migration ("DB Graph Importer")**: automatically detects property types during migration and offers choices for how to handle tags; documented limitation — "blocks with multiple code snippets, embeds, or quotes only import the first instance" ([docs/db-version.md](#)).
- A dedicated audit doc, [docs/og_import_graph_cases.md](#), catalogs logseq/db-test issues labeled import as of June 2026 across seven problem categories — block-identity conflicts, forward references (a block referencing a target defined in a later-imported file), journal-filename legacy references, mixed deadline/scheduled timestamp formats, linked external PDFs and missing local assets, and parser edge cases (empty files, huge flat blocks, self-referencing blocks) — with 17 regression tests covering them ([docs/og_import_graph_cases.md](#)).
- Concrete import-fidelity bugs open in db-test as of Aug 2026: PDF annotations on linked (non-asset) PDFs are silently dropped on MD→DB import (open, [#923](#)); embedded asset alt text is lost on file→DB import (open, [#1081](#)); pasting large markdown content can set an invalid :block/pre-block? flag that later breaks RTC sync (open, [#775](#)); EDN import can error when importing a tag already created by a page import (open, [#958](#)).
- **The Markdown Mirror** (new, shipped ~May/June 2026) is the closest thing to an "export" that stays live — see §10, since its real purpose is answering the lock-in complaint rather than being a data-interchange format; ADR 0016 is explicit that it is **not** "a complete database export despite including property drawers" ([ADR 0016](#)).

10. Storage format & data ownership

Source-of-truth facts (~logseq/graphs/<name>/db.sqlite, single-writer lock, etc.) are in the sibling doc's §1 — not re-derived.

- **COMMUNITY SENTIMENT** (per a WebSearch synthesis of [discuss.logseq.com](#) threads, attribution to specific individual posters not independently re-verified here): users describe the SQLite move as a step away from "portable plain-text files," worry the two graph formats (OG vs DB) are not interoperable, and characterize even a working export/import path as a "soft lock-in" because it requires a deliberate manual step rather than being the native format. Treat this framing as forum sentiment, not an official Logseq claim.
- **Official response — the split announcement**: the team frames DB graphs as still "local-first" — the SQLite file lives "exclusively on your local device, just like the file version," and even with Sync enabled, data is "encrypted locally on your device" before upload, with each device keeping its own local copy ([logseq.io](#), "[Big update: Logseq is splitting into two versions](#)").
- **Official response — the Markdown Mirror feature (the concrete product answer to the plaintext complaint)**: an Electron-only setting that renders each page to a .md file under mirror/markdown/{journals,pages}/ inside the graph folder, incrementally, "to provide desktop users with readable Markdown files inside their graph directory for external tool integration, backup, indexing, and inspection outside Logseq itself." It is explicitly **one-way**: "Editing files in mirror/markdown/ does not update the graph" — the SQLite DB remains sole source of truth, mirror files are debounced/lagging and get overwritten on next edit, ambiguous refs are unresolved in the mirrored text, and it is unavailable on browser/mobile builds ([ADR 0016](#)). The roadmap separately still lists "enabling reliable two-way sync with Markdown files" and "treating each Markdown file as a single block in the database" as *future*, unshipped exploration ([logseq.io split announcement](#)). The team's own May 2026 update lists "Two-way markdown mirror editing" as still "Coming Soon," i.e. not shipped as of 2.0.1 ([discuss.logseq.com](#), [May 16 2026 update](#)). There is also at least one open regression on the mirror itself: "Failed to regenerate Markdown Mirror: Error: Maximum call stack size exceeded" (closed after fix, [db-test #978](#)), evidence the feature is young and was actively being hardened around the 2.0.1 cut.
- **Synthesis** (mine, not sourced to one document): the practical honest answer to "do I still own my files" is: no, not natively — SQLite is the live source of truth and the file version is now a second, frozen product (Logseq OG) — but Logseq shipped a concrete (if one-way, desktop-only, lossy-on-ambiguous-refs) mitigation within months of the 2.0 beta, which is more than a purely rhetorical response to the controversy.

11. Plugins/extensibility

- **Plugin API status**: exists and is actively used for DB graphs via the @logseq/libs SDK (npm, "0.2.* SDK for plugin development," version 0.2.11 cited by a forum responder) plus a parallel CLJS SDK ([docs/db-](#)

[version.md](#); forum thread [discuss.logseq.com, "Logseq DB plugins"](#) — responder's staff status not confirmed, treat the SDK-version claim as corroborated by npm but the framing/tone as community).

- **Marketplace compatibility mechanism:** the `logseq/marketplace` package manifest format has explicit boolean flags `supportsDB` and `supportsDBOnly` ("Whether the plugin supports database graph" / "Whether the plugin only supports database graph," both default `false`) ([logseq/marketplace README](#)) — i.e. DB-graph support is opt-in per plugin, not automatic, and most of the marketplace's several-hundred plugins were written for the file version and are not DB-compatible by default. `db-version.md` claims "Over 65 plugins support DB graphs" as of its April 2026 snapshot ([docs/db-version.md](#)) — a small fraction of the marketplace catalog.
- **"Famously incomplete at beta":** confirmed by open `db-test` issues specifically in the plugin surface as of Aug 2026: `addPropertyValueChoices` "silently fails: worker-side assert requires `cljs` uuids that JS callers cannot supply" (open, [#1032](#)); `appendBlockInPage` cannot add properties and `upsertBlockProperty` is "limited to plugin-defined string properties" (open, [#294](#)); the `name` option in `logseq.Editor.upsertProperty` not working (open, [#987](#)); HTTP-API property writes landing under the wrong namespace vs. documented `:plugin.property._api` (open, [#1051](#)); custom block renderers, improved plugin discovery (search by description), and plugin thumbnail icons were only added in the May 2026 update, i.e. post-beta polish, not day-one ([discuss.logseq.com, May 16 2026 update](#)). For security, "only plugins configured with no 'effect' are usable" for web-accessible plugins ([db-version-changes.md](#)).
- **CLI/MCP as new extensibility surface:** per the sibling doc, a new private OCaml/Melange `cli/` (binary `logseq`) ships `mcp-server` and `skill show` subcommands, i.e. Logseq is positioning a local CLI + MCP server as first-class extensibility alongside (eventually replacing) the JS/CLJS in-app plugin API — see [sibling doc §3](#). `db-version.md` corroborates an MCP server offering "batch creates/edits, search, tag/property/page management, and validation" ([docs/db-version.md](#)).

12. Whiteboards, flashcards/SRS, PDF annotation

The row most likely to have real gaps — confirmed:

- **Whiteboards: removed.** "Whiteboards have been removed as a feature and will hopefully be available as a plugin"; the built-in `/draw` command is similarly gone, "hopefully" to become a plugin. This applies to DB graphs; whiteboards still function in the frozen Logseq OG/markdown product ([db-version-changes.md](#)). A built-in `Whiteboard` class name still exists in the datascript schema per the sibling doc's class list, but the feature/UI built on it is absent in the DB version as shipped.
- **Flashcards: reimplemented, not migrated, still buggy.** "Flashcard reimplementation: new algorithm incompatible with previous versions; no property/SRS data imported" from OG graphs ([db-version-changes.md](#)). `db-version.md` describes Cards as a tag-driven system using a spaced- repetition algorithm ([docs/db-version.md](#)). Multiple flashcard bugs remain open in `db-test` as of Aug 2026: deleting a `#card` tag while viewing the flashcard crashes Logseq (open, [#1088](#)); unable to filter flashcards to review (open, [#495](#)); adding a "State" property to a non-node-typed tag "breaks flashcards and sends answering into a loop" (open, [#524](#)). Separately, a pre-DB-version issue on the main repo records that the file-version SRS algorithm was acknowledged faulty and the team's fix path was to correct it only in the DB version rather than backport ([logseq/logseq #8890](#)).
- **PDF annotation: changed shape, partially working, import-lossy.** Annotations are now tagged entities ("annotation tags") displayed by default beneath the asset block rather than requiring a separate PDF-viewer view, "allow[ing] annotations to be viewed across pdfs and to have custom views" ([db-version-changes.md](#)). Import fidelity is a known weak point: annotations on linked (non-asset) PDFs are silently dropped on MD-graph import (open, [db-test #923](#)); importing a legacy graph with PDF annotations could previously fail outright with "Cannot store nil as a value" (closed/fixed, [db-test #1008](#)); "PDF Highlights not working correctly" and "PDF highlighting not working anymore" were both filed and since closed ([db-test #650](#), [#529](#)) — i.e. PDF annotation round-tripped through multiple broken states before stabilizing, and cross-graph-type import remains lossy as of Aug 2026.

13. Calendar/journals

Storage facts (journal pages tagged `:logseq.class/Journal`, `:block/journal-day` as integer `yyyyMMdd`) are in the sibling doc's [§5](#). UX additions:

- Journal pages are created automatically and accept natural-language date input for navigation/creation ("Today," "Next Friday") ([docs/db-version.md](#)).
- No primary source describes a changed calendar-grid UI for DB graphs beyond this; `db-version-changes.md` explicitly notes "No specific changes detailed beyond unified node system" for journals in its diff-from-OG framing.

14. Platforms

- **Desktop:** Electron, current shipping beta channel is 2.0.1 (2026-07-13); a rolling `nightly` build channel also exists (latest tag 20260819, per GitHub Releases) for pre-release DB-version work (github.com/logseq/logseq/releases).
- **Web app:** browser/WebView builds run on SQLite WASM over OPFS (per the sibling doc's §1), confirmed as a currently-live storage platform (`browser.cjs`), not a future plan.
- **Mobile — DB-graph support is pending/partial, not fully shipped:**
 - iOS: an **invite-only** native app exists with a mobile-first five-tab UI (Home, Graphs, Capture, Go To, Search), voice capture, and external-app share integration ([docs/db-version.md](https://docs.logseq.com/db-version.md)).
 - Android: "New native implementation under development. Alpha testing not yet opened" as of the doc's snapshot ([docs/db-version.md](https://docs.logseq.com/db-version.md)).
 - The public roadmap corroborates this is still in motion: "Android native experience" is an assigned, in-progress roadmap item, and general "Native mobile apps" are listed as a future goal rather than done ([logseq.io public roadmap](https://logseq.io/public-roadmap)).
 - `docs/db-version.md` itself notes mobile apps "lack complete desktop feature parity" as a known limitation.
 - Net: as of Aug 2026, DB-graph mobile access is real but explicitly beta/invite-gated (iOS) or pre-alpha (Android) — not a mainstream shipped mobile experience yet.

15. Publishing

Publishing for DB graphs exists as a paid, Sync-gated feature, contrary to the row's "likely absent" prior:

- **Logseq Publish** requires a Sync account and is paid; it produces password-protectable, read-only pages hosted at `logseq.io`, with Cloudflare-backed public routes for cross-graph discovery: `/tag/TAG`, `/u/USER`, `/graph/GRAPH-UUID`; a self-hosting path (via Cloudflare setup) is also documented ([docs/db-version.md](https://docs.logseq.com/db-version.md)).
- The official public roadmap listed "Page publishing" with a target of "end of 2025," i.e. this shipped ahead of the July 2026 desktop beta as a Sync- tier feature rather than being introduced with 2.0.1 ([logseq.io public roadmap](https://logseq.io/public-roadmap)).
- The May 2026 team update reports a **privacy fix** — "Protected pages now stay hidden from public listings, with configurable self-hosted publish server support" — implying Publish had a real privacy bug (previously password-protected pages could still surface in public listings) that was patched shortly before the 2.0.1 desktop beta (discuss.logseq.com, [May 16 2026 update](#)).
- No equivalent publishing story exists for Logseq OG/file graphs in any source found — Publish is DB-graph/Sync-only.

16. Licensing / governance / maintenance

- **License:** `logseq/logseq` remains **AGPL-3.0**, confirmed both via GitHub's license API detection and the repo's `OWN LICENSE.md` (github.com/logseq/logseq, license badge references [LICENSE.md](#)). No license change accompanied the DB pivot.
- **The fork/split is real and structural, not just marketing:** a separate repository `logseq/og` ("Logseq og (file version)") exists, created 2025-12-25 — i.e. the OG split was being prepared on GitHub roughly seven months before the 2.0.1 DB beta shipped (github.com/logseq/og). The official announcement states Logseq OG will get "security fixes and patches" and "Electron and dependency upgrades" but not new features going forward, with all future feature investment going to the DB version (logseq.io, "[Big update: Logseq is splitting into two versions](#)"). A dedicated `logseq/db-test` repository ("Used for Database version test") hosts the DB-version issue tracker separately from the main repo (github.com/logseq/db-test).
- **Release cadence:** checked the full GitHub Releases history via the API (`gh api repos/logseq/logseq/releases`). The pre-2.0 file-version product shipped **very frequently** — roughly 90+ tagged "Beta Testing" desktop releases from 0.3.8 (2021-09-11) through 0.10.15 (2025-12-01), often multiple per month in the 0.6–0.9 era. Then a long gap: nothing tagged between 0.10.15 (2025-12-01) and 2.0.1 (2026-07-13) — over seven months with **zero** tagged desktop releases while the DB rewrite was finished. Since 2.0.1, as of 2026-08-20 (five-plus weeks later) there is **exactly one** tagged 2.0.x release — no 2.0.2 yet — with a separate `nightly` unstable channel (latest 20260819) absorbing ongoing changes instead of dot releases (github.com/logseq/logseq/releases). Read together with the frequent-but-small forum "What's New with Logseq DB" posts (e.g. April 26 and May 16, 2026 — discuss.logseq.com), the team is shipping continuously to the nightly/forum-announcement cadence but has been conservative about cutting a second tagged desktop beta release in the five weeks after 2.0.1.
- **Team focus shift:** corroborated directly by the split announcement's framing that maintaining two architectures meant "every feature, bug fix, and UX improvement must be built twice, resulting in slower

development," and that the split's purpose is to let the team commit fully to the DB version's roadmap ([logseq.io](#), "Big update: Logseq is splitting into two versions").

Pros / Cons of the DB architecture

Each point is grounded in a primary source; synthesis-only points are labeled.

Strengths

- **Typed, first-class data model.** Properties, classes, and tags are real datascript entities with declared types and closed-value choices, not regex-parsed `key:: value text` — enables the visual query/table/gallery Views layer and bulk operations that were structurally impossible on free-text properties ([docs/db-version.md](#); §1, §5 above).
- **Performance at scale.** The team's own framing: "The application performance is better — loading faster, handling larger graphs and large tables" ([db-version-changes.md](#)), and the roadmap cites 50k-page large-graph support as a sync/DB target ([logseq.io split announcement](#)) — directly answering OG's well-known large-graph slowness (**synthesis**: consistent with, though not itself proof of, community reports of multi-minute OG load times on large graphs).
- **Real sync/collaboration substrate.** A structured op-log (`client_ops/sync_meta` in `client-ops-db.sqlite`) with checksums and ordered pending-upload replay is a materially more sync-able foundation than diffing markdown files ([ADR 0015](#)).
- **No parse fragility.** Content lives as datoms, not as markdown text that must be re-parsed on every read/write — the whole class of "regex missed an edge case in `key:: value` parsing" bugs the sibling doc's importer concerns are built around simply doesn't apply inside Logseq itself (**synthesis**, following from the sibling doc's data model description).
- **Concrete, if partial, answer to lock-in.** The Markdown Mirror ships a real (if one-way, desktop-only) live markdown projection within months of the beta, rather than leaving the plaintext complaint unaddressed ([ADR 0016](#)).

Weaknesses

- **Data-lock-in perception is real and team-acknowledged as a tradeoff.** SQLite is the sole live source of truth; the sanctioned interchange format (EDN) requires an explicit export step and specialized tooling to read (see sibling doc §3/§4) — a materially higher barrier than "open the folder in any text editor," which is what OG offered natively.
- **Markdown export is lossy by design, not just by bug.** The in-app markdown export carries a literal `"TODO: indent-style and remove-options"` in its own source and cannot represent DB-only constructs (typed properties as entities, class/ontology, closed values, Views) — see sibling doc §2. The Markdown Mirror is explicitly *not* a substitute: it drops ambiguous refs, excludes property pages, and is one-way ([ADR 0016](#)).
- **Plugin ecosystem regression at beta.** Marketplace compatibility is opt-in per plugin (`supportsDB` flag, [logseq/marketplace README](#)), and `db-version.md`'s own count — "Over 65 plugins support DB graphs" — is a small fraction of the multi-hundred-plugin marketplace ([docs/db-version.md](#)). Multiple plugin-API calls (`addPropertyValueChoices`, `upsertBlockProperty`, `upsertProperty`, HTTP-API property namespacing) have open correctness bugs in `db-test` as of Aug 2026 (§11 above, with issue links).
- **Feature regressions at 2.0.1, not just format changes.** Whiteboards were removed outright rather than migrated; flashcards were reimplemented with no data migration from OG graphs (old SRS history is not imported); PDF annotation import from legacy graphs is lossy for linked (non-asset) PDFs (§12 above, with issue links) — these are real feature debt, not merely "different UX."
- **Fork/community friction is structural, team-acknowledged, and resource-splitting.** The team's own words: two architectures meant "every feature, bug fix, and UX improvement must be built twice" — the fix (splitting into Logseq OG vs Logseq) is itself evidence the community was split enough by architecture that the team judged serving both simultaneously unsustainable ([logseq.io split announcement](#)). The eight-plus-month gap in the desktop release history between `0.10.15` (2025-12-01) and `2.0.1` (2026-07-13) — during which OG effectively froze — is itself the visible cost users paid for the rewrite (§16 above).

Rows with the thinnest primary-source coverage

Flagged honestly rather than overstated:

- **§6 (knowledge graph/backlinks)**: solid on the graph-*visualization* rebuild (Graph View V2), but no primary source found describing any algorithmic change to backlink discovery/ranking itself.
- **§8 (sync pricing)**: no primary-source figure found for DB-version Sync/RTC/"Logseq Pro" pricing; only third-party aggregator blogs had numbers, which were deliberately excluded per the sourcing rules.
- **§13 (journals)**: thin by nature — the primary sources themselves say there's little beyond the unified node system to report for journals in the DB version.

"Tine" and the Logseq file-format fork question (as of 2026-08-20)

Identity verdict — read this first

A real, actively-developed project named **Tine** exists and is genuinely positioned as a Logseq-compatible, file-based alternative: github.com/martinkoutecky/tine (website tine.page). **But it is explicitly, repeatedly NOT a fork** in the git-lineage sense — the maintainer states in the README and on the site that it is "an independent reimplementation, not a fork" with "original Rust and SolidJS containing no Logseq source." It targets Logseq's on-disk Markdown format and adapts parts of Logseq's outliner CSS, which the maintainer says makes it "a derivative work for licensing purposes" (hence inheriting Logseq's AGPL-3.0 license), but there is no shared codebase, no forked git history, and no PR relationship to `logseq/logseq`.

If the premise behind the ask was "a community fork that continues file-based Logseq after the DB-version pivot," Tine is the closest known real match — but the correct one-line correction is: **it's a from-scratch, Logseq-format-compatible outliner, not a fork**. No evidence was found of a project literally forked from the `logseq/logseq` repository and renamed "Tine." I searched GitHub, Logseq's own forum (discuss.logseq.com), and Hacker News (via Algolia) for any announcement of a fork named Tine — none exists. Two unrelated repos surfaced in general "logseq fork" searches (`Team-R3SET/logseq-fork`, `stevelab1/logseq-fork-2026`) — both are low-signal/unclear-purpose repos with no connection to "Tine" and are not discussed further here. No unrelated "Tine 2.0"/Tine Groupware (PHP CRM/groupware suite) or npm/design-tool namesake collision was found in the Logseq context — the name doesn't appear contested.

Primary sources fetched 2026-08-20 (today): GitHub repo page + API, raw README, `docs/FEATURES.md`, `CONTRIBUTING.md`, releases page, tine.page homepage, tine.page/compare.html.

1. Provenance

- **Repo created:** 2026-06-24 (GitHub API, `created_at: 2026-06-24T21:59:31Z`) — under two months old as of this research.
- **Not forked from any Logseq commit/version.** GitHub's own fork flag confirms this: `"fork": false` in the repo API response. It is a ground-up rewrite that targets Logseq's *file format* (`journals/`, `pages/`, `assets/`, `logseq/config.edn`), not its codebase.
- **Maintainer:** Martin Koutecký (GitHub handle `martinkoutecky`), a single named human maintainer. Contributors per GitHub API: 3 total (`martinkoutecky`, `da5nsy`, `EllisMorrow`), overwhelmingly maintainer-driven.
- **Notable authorship claim:** tine.page states "Tine is built by Claude Code (AI) under human direction and review by Martin Koutecký," with the creator "emphasizing transparency about AI-assisted development while maintaining human oversight" (tine.page, fetched 2026-08-20). `CONTRIBUTING.md` corroborates: implementations are "often AI-assisted" during review, after a human-approved design proposal.
- **Stated motivation** (maintainer's own words, README/tine.page): "Logseq's UI is Electron + DataScript with heavy re-rendering, and it gets sluggish on large graphs." The fix chosen was not forking Logseq's code but "a ground-up rewrite: a small native shell (Tauri/WebKitGTK), a pure-Rust core for parsing and indexing, and a fine-grained reactive frontend (SolidJS)." This is a **performance/architecture complaint, not a DB-version-migration protest** — the DB-version pivot is not cited anywhere in the primary sources as the motivating grievance. The site does note Tine intentionally excludes "database-version Logseq support," i.e., it targets file/Markdown Logseq by design, but frames this as staying focused rather than reacting to the DB pivot.
- **License:** GNU AGPL-3.0-only ([repo license file](#), confirmed via GitHub API `license.spdx_id: AGPL-3.0`), inherited because the project is "a derivative work for licensing purposes" of Logseq's format/CSS.
- **Governance:** single-maintainer, closed-PR model for code (see §4).

2. Divergence from Logseq OG — and the core premise check

Core premise confirmed: Tine is genuinely file/Markdown-based and interoperable with Logseq's on-disk graph. Per tine.page/compare.html and the README, it "Reads and writes the *same* markdown graph as Logseq — swap between the two on the same files," reading `.md/.org` files under `journals/`, `pages/`, `assets/`, and `logseq/config.edn`, with no import/export step required. Files stay plain Markdown that the user fully owns.

Features added beyond Logseq OG (per [FEATURES.md](#) and [compare.html](#)):

- **Sheets:** 2-D grids, typed field tables, Kanban/tag boards, formula columns, built on plain Markdown blocks.
- **Split view:** independent panes, tabs, per-pane history.
- **Built-in browser-style tabs,** persistent search-as-workspace tabs.

- **Global quick-capture** via desktop hotkey from any application.
- **Task carry-forward**: rolls unfinished tasks forward by configurable timeframe.
- **Native PDF annotation** with highlights, area highlights, reader themes — "Logseq has no native PDF export — only a community plugin" (compare.html).
- **Native PDF export** per page.
- Performance: native Rust/Tauri/SolidJS core instead of Electron/DataScript, explicitly targeting large-graph sluggishness.
- Android: native arm64 app (Tauri v2) as of v0.4.0, editing the same Markdown files, sideloaded APK.

Deliberately removed/out of scope (compare.html, "Deliberately Out of Scope"): whiteboards/canvas as a native tool (external drawio/Excalidraw SVG round-trip supported instead), flashcards/spaced repetition, Logseq or Obsidian plugin-API compatibility, real-time multi-user collaboration, and DB-version Logseq support.

Query engine is intentionally scoped down: "supports everyday queries but not raw Datalog with arbitrary entity joins or custom rules" — unsupported Datalog clauses are flagged rather than silently ignored (compare.html).

Graph view (node-link visualization) is **not yet implemented**; only "local-neighborhood visualization is planned" (compare.html).

3. Sixteen-row taxonomy

Row	Tine status	Source
Note model	Inherited concept (blocks, outline, <code>key:: value</code> properties, <code>id::</code>), full Markdown/Org round-trip. Unchanged in spirit from Logseq OG.	FEATURES.md
Editor	Full Logseq-style keyboard semantics (Enter/Tab/Shift-Tab/Backspace), block zoom/collapse, wrap-then-type, callouts, sanitized inline/block HTML — plus additions (typographic replacement, calculator slash command). Not a plain "inherited, unchanged" row — meaningfully re-implemented and extended.	FEATURES.md
References	Page refs, hashtags, block refs (<code>(id)</code>), embeds <code>{{embed}}</code> , autocomplete, linked/unlinked-reference panels, hover previews — parity claimed with Logseq.	FEATURES.md, compare.html ("Full Parity ✓" list)
Tasks	TODO/DOING/DONE/NOW/LATER/WAITING/CANCELED states, two workflows, priorities, LOGBOOK time-tracking (OG-compatible), SCHEDULED/DEADLINE, recurring tasks — plus Tine-only task carry-forward.	FEATURES.md
Queries	Query DSL parity for common cases (<code>task</code> , <code>priority</code> , <code>property</code> , <code>between</code> , etc.) plus visual/no-code query builder; explicitly not full raw Datalog with joins/custom rules.	compare.html ("Partial/Scoped 🟡: Queries")
Graph (visualization)	Not implemented. "Not yet implemented; local-neighborhood visualization is planned."	compare.html
Search	Ctrl+K quick switcher, full-text, search syntax (phrases/regex/exclusions), ranking, in-page find, persistent search-as-workspace. Extended beyond OG.	FEATURES.md
Sync	No cloud sync backend of its own; filesystem watcher (inotify/poll), never silently overwrites externally-changed files, and dedicated conflict-merge UI for Syncthing/Dropbox <code>*.sync-conflict-*</code> files (block-by-block diff by <code>id::</code>).	FEATURES.md ("Sync & Conflict Resolution": "Absent: Cloud sync backend; native Syncthing integration.")
Import/export	Static HTML publish (own <code>public::</code> -page exporter, offline full-text search), PDF export per page, Markdown/Org copy-export. No direct integration with Obsidian Publish/Notion-style platforms.	FEATURES.md
Storage	Same flat-file Markdown/Org graph as Logseq OG (journals/, pages/, assets/, logseq/config.edn); namespaces stored as flat <code>parent__child.md</code> files, not real folders — this matches Logseq OG's own convention.	FEATURES.md, tine.page

Row	Tine status	Source
Plugins	Original Logseq plugins do NOT work. Tine ships its own experimental WASM-based plugin API (v0.2) with no DOM/file/network access, and a separate token-based theme API — explicitly stated as no JS-plugin compatibility: "Absent: JavaScript plugin API; Logseq @logseq/libs or Obsidian API compatibility."	FEATURES.md ("Plugins & Extensions")
Whiteboards	No native canvas tool. Supports external drawio/Excalidraw round-trip (/drawio creates an editable SVG asset, opens in the external editor).	FEATURES.md ("Whiteboards & Diagrams")
SRS (spaced repetition)	Absent. <code>{{cloze}}</code> renders only in a "degraded" click-to-reveal form with no scheduling/SRS engine behind it.	FEATURES.md
PDF annotation	Present and a stated differentiator: zoomable/virtualized PDF pane, text/area highlights, reader themes, outline nav — richer than Logseq OG's plugin-only PDF support.	FEATURES.md, compare.html
Journals	Multi-day continuous feed, journal templates, agenda view, calendar with markers, configurable date formats — parity plus extensions (carry-forward, agenda).	FEATURES.md
Platforms/mobile	Desktop: Linux (primary/best-tested), macOS and Windows ("newer builds"). Mobile: Android only (native Tauri v2 arm64, sideloaded APK, Play Store/F-Droid "planned"). No iOS — site directs iOS users to "Logseq mobile or fastlog" instead.	tine.page, FEATURES.md ("Mobile (Android)": "Absent: iOS app (being scoped)")
Publishing	Static HTML export with offline search (Fuse.js) and PDF export; no scheduled or multi-platform publishing.	FEATURES.md
Licensing/governance	AGPL-3.0-only, single primary maintainer, code contributions explicitly not accepted as PRs ("Tine does not merge externally-written code into the app" — contributors submit design proposals/specs instead; docs/typo PRs are the exception).	CONTRIBUTING.md

4. Viability signals

- **Stars:** 311. **Forks:** 22. **Open issues:** 112. (GitHub API, fetched 2026-08-20.)
- **Contributors:** 3 total per GitHub API (martinkoutecky, da5nsy, EllisMorrow) — effectively a solo/AI-assisted project with light outside involvement, consistent with the "no code PRs" contribution model in CONTRIBUTING.md.
- **Commits:** ~2,797 on `master` (per repo page), against a repo only created 2026-06-24 — very high commit velocity for under two months, plausibly reflecting the stated AI-assisted implementation workflow.
- **Release cadence:** near-daily in the most recent stretch — v0.6.90 (Aug 5), v0.6.91 (Aug 7), v0.6.92 (Aug 11), v0.6.93 (Aug 12, 2026, latest at fetch time) — after a slightly slower mid-July run (v0.6.0–v0.6.5, Jul 18–22). [Releases page](#).
- **Status self-description:** "Usable daily-driver; not yet version 1.0" (repo page).
- **Prebuilt binaries:** Linux (ApplImage, .deb, .rpm), macOS (.dmg), Windows (.exe + portable .zip), Android (sideloaded arm64 APK). No iOS build. All via GitHub Releases — no app-store distribution yet.
- **No forum/HN footprint found:** an Algolia HN search for "Tine Logseq" and "tine.page" returned no matching Show HN/story threads, and a discuss.logseq.com search for "tine" returned nothing. This suggests the project has not yet had a wide public launch moment (no HN front-page post, no Logseq-forum announcement thread) — it may be growing through direct GitHub/word-of-mouth discovery only. Community awareness signals (stars/forks) are still small.

5. Honest pros/cons

Tine vs staying on Logseq OG (file-based):

- *Pro:* faster on large graphs (native Rust/Tauri core vs Electron+DataScript); adds PDF annotation, sheets/kanban, split-view, and task carry-forward that Logseq OG lacks natively; files stay byte-compatible so switching is reversible any time.
- *Con:* no original Logseq plugin ecosystem (JS plugin API not supported — a real loss if you depend on community plugins); no native whiteboard; no SRS/flashcards; no graph visualization yet; single-maintainer/

AI-assisted project barely two months old with only 311 stars — much higher abandonment/instability risk than an established Logseq OG install; no iOS.

Tine vs moving to Logseq's new DB version:

- *Pro:* keeps notes as plain Markdown files under full user control (no SQLite/datascript lock-in — see this repo's own [research-logseq-db-format.md](#) for how opaque the DB-version storage is); avoids the DB version's migration risk and format churn entirely; arguably faster than either Logseq variant given the from-scratch native core.
- *Con:* the DB version gets Logseq's own ongoing investment (official product, presumably eventual mobile/sync maturity, official plugin ecosystem evolution); Tine explicitly does not support the DB format at all, so it's not a bridge to that world — picking Tine is a bet on staying file-based indefinitely, maintained by one person, not a hedge against Logseq's own direction.

TyLog feature inventory (v0.3.0+92, 2026-08-20)

Ground truth for the PKMS competitive audit (docs/audit-pkms-comparison.md). Gathered by codebase exploration over lib/, packages/, docs/, spec/. TyLog is a deliberately scoped Typst-first workspace prioritizing local-first data ownership, plaintext storage, and depth over breadth.

Note model

Feature	Description	Where	Maturity
Typst file storage	Notes are .typ files with valid Typst syntax	daily/, notes/, projects/, articles/	Shipped
Note kinds	note, daily, project, article, research; extensible (person, place, organization)	spec/tylog-format-v1.md, scanner.dart	Shipped
Tags	Freeform tags + synonym normalization (_system/tag-synonyms.json)	packages/tylog_core/src/scanner.dart	Shipped
Aliases	Alternative display names	NoteRef.aliases	Shipped
Properties	Custom key-value metadata	NoteRef.properties, property_select_chip.dart	Shipped
Daily/journal notes	daily/YYYY/MM/YYYY-MM-DD.typ	journal_feed.dart, month_calendar.dart	Shipped
Note dates, project refs	Temporal + project context fields	NoteRef.date, NoteRef.project	Shipped

Editor

Feature	Description	Where	Maturity
Block-level editing	Edit individual blocks without full source	controlled_editor.dart, TyLogReadView	Shipped
Preview/Source/Split	Three editor modes	editor_panel.dart, work_surface.dart	Shipped
Magic (/) actions	20+ quick-insert commands	app_mobile.dart applyMagic(), MagicAction	Shipped
Rich formatting	Bold/italic/strike/underline/mono/highlight (4 colors) via Typst	app_mobile.dart	Shipped
Headings, tables, equations	Via magic menu; LaTeX-style math	MagicAction.*	Shipped
Wikilinks + autocomplete	[[Note Display]] → #tylog.ref-note(); note/project/person suggestions	editor_autocomplete.dart	Shipped
Autosave + history	Atomic per-note saves; 100-entry session undo/redo	controlled_editor.dart	Shipped
Citations	From _system/bibliography.yml (BibTeX/BibLaTeX)	bibliography.dart	Shipped
Attachments, reports	File/image refs; filtered note reports (date/status/tag/kind)	MagicAction.attachment, report.dart	Shipped
ABSENT: markdown storage, arbitrary WYSIWYG, vault-level undo	Intentional (bulk changes snapshot to .tylog/undo/)	—	N/A

Tasks

Feature	Description	Where	Maturity
Statuses, priorities	todo/doing/done/cancelled; low/normal/high/urgent	TaskRef, scanner.dart	Shipped
Due + scheduled dates	ISO 8601, separate fields	TaskRef.due/.scheduled	Shipped
Reminders	Local notifications	task_scheduler.dart	Shipped
Recurrence	RRULE via rrule package	TaskRef.recurrence	Shipped
Time tracking	Clocked sessions with runaway filtering	TaskRef.clocked, ClockEntry	Shipped
Tags, project, assignees, dependencies, completion history, custom properties	Full task data model	TaskRef	Shipped
Task views	Library > Tasks, Today agenda, status/priority filters	work_surface.dart	Shipped

Knowledge graph / linking

Feature	Description	Where	Maturity
Backlinks + forward links	Reverse link index; outgoing per note	<code>VaultIndex.backlinksByTarget</code> , <code>linked_references.dart</code>	Shipped
Five graph modes	Concept map, Focused (1 - hop), All files (LOD), Timeline, Voronoi treemap	<code>graph.dart</code> , <code>voronoi_view.dart</code>	Shipped
Community detection	Louvain-like clustering	<code>computeCommunities</code>	Shipped
Edge types	link/citation/tag/read with toggles	<code>GraphEdgeKind</code>	Shipped
Auto-related sections	LLM-generated or title/tag/citation-matched ("Relink vault")	<code>app_mobile.dart</code> <code>stripAutoRelated()</code>	Shipped
ABSENT: block refs/transclusion	Note-level references only, by design	—	N/A

Search & indexing

Feature	Description	Where	Maturity
Full-text search	Tokenized, gzipped JSON index (<code>_index/search-index.json.gz</code>)	<code>search_index.dart</code>	Shipped
Worker-isolate indexing	Background indexing off the UI isolate	<code>vault_worker.dart</code>	Shipped
Saved searches	Named presets with tag/status filters	<code>saved_searches.dart</code>	Shipped
Search filters	<code>kind/tags/status/date/article-status</code>	<code>knowledge_screen.dart</code>	Shipped
Fallback parser + validation	Safe parse when Typst fails; problem reporting	<code>scanner.dart</code> , <code>validation.dart</code>	Shipped
ABSENT: query language	No <code>{{query}}</code> /datalog/SQL; saved searches + report blocks instead	—	N/A

Sync

Feature	Description	Where	Maturity
Nextcloud WebDAV sync	Polling, conditional transfers, SHA-256 change detection	<code>nextcloud_sync.dart</code> , <code>path_sync.dart</code>	Shipped
Conflict resolution	Manual: text edit, binary choice, delete-vs-edit; snapshots; pending conflict suspends auto-sync	<code>nextcloud_sync/conflicts.dart</code>	Shipped
ETag safety, atomic writes	Never overwrite on remote change; temp/flush/rename	<code>nextcloud_sync.dart</code> , <code>vault_storage.dart</code>	Shipped
Sync dashboard	Diagnostics, transfer totals	<code>sync_dashboard.dart</code>	Shipped
Android SAF	Persistent folder access	<code>vault_registry.dart</code>	Shipped
ABSENT: 3-way merge, E2EE sync, version history, multi-user	Manual resolution; plaintext over HTTPS	—	N/A

Import / export

Feature	Description	Where	Maturity
Logseq + Obsidian vault import	Auto-detect; pages→notes, journals→daily, TODO→tasks, wikilinks, assets, import report	<code>vault_import_flow.dart</code> , <code>tylog_import_core</code>	Shipped
Logseq DB (2.0) import	Via EDN export, transpile to file pipeline	<code>docs/superpowers/plans/2026-08-20-logseq-db-import.md</code>	Planned
Markdown article import	Single articles	<code>markdown_article_import.dart</code>	Shipped
PDF export	Any note/report via typst compile; .typ + .pdf siblings in outputs/	<code>report.dart</code>	Shipped
Bibliography	BibTeX/BibLaTeX	<code>bibliography.dart</code>	Shipped
ABSENT: HTML export	PDF only	—	N/A

Library / articles

Feature	Description	Where	Maturity
Article pipeline	Status stages unread→skimmed→read→extracted→cited; ratings; reading log to daily notes	vault.dart, app_mobile.dart _logReading	Shipped
Reading mode	Adjustable typography, night mode (per - device prefs)	reading_mode.dart	Shipped
External LLM producer	article - pipeline repo writes Typst articles into the vault	docs/tylog - ecosystem.md	Shipped (external)
ABSENT: Zotero integration	Static .yml only	—	N/A

Calendar

Month grid with journal/task markers, date references as calendar entries, task dues, recurring tasks, native date picker (calendar_tab.dart, month_calendar.dart, VaultIndex.calendar). Shipped.

Platforms & storage

Android (release-grade, real-vault tested on Huawei P30), macOS (release, auto-updater), iOS host for dev, Linux CI-only. Multi-vault, SAF, plaintext storage by design; age-encrypted backup designed (docs/age-encrypted-backup.md); background sync service (WorkManager). ABSENT: in-app encryption, E2EE.

PDF / typesetting (TyLog's unique axis)

Typst-native notes compile directly to typeset PDF; reproducible reports (.typ + .pdf); themable (_system/theme.typ, export.typ); native math; bibliography/citations. No markdown app matches this without export toolchains.

Extensibility

Custom properties and note kinds only. ABSENT by choice: plugin API, query blocks, flashcards/SRS, whiteboards, kanban, real-time collaboration, in-app AI/RAG.

Notable Logseq features TyLog lacks (for the audit's gap list)

Block refs/{{embed}}, plugin ecosystem, query language, SRS/flashcards, whiteboards, kanban, RTC/multi-user, sharing/permissions, version history, 3-way merge, encrypted sync, HTML publish, favorites/pins, recycle-bin UI, PDF annotation.